WHITE-LABEL BUSINESS PLAN

Single-Vendor Grocery / Supermart Delivery Platform

Using Enatega Single Vendor Frontend + Custom AI-Built Backend

Goal: Learn white-labeling of software products using AI tools, build a production-ready hyperlocal delivery platform, and sell branded solutions to local businesses (grocery stores, supermarts, coaching centers, schools) to generate recurring revenue.

Prepared: 02 October 2026

Stack: Supabase • Vercel • GitHub • Razorpay • Cloudinary

AI Tools: Google Antigravity + Claude Code

1. Executive Summary

This plan outlines a complete path to launch a white-label hyperlocal grocery delivery platform. We will use the open-source Enatega Single Vendor frontend (Customer App, Rider App, and Admin Dashboard) and reverse-engineer + rebuild a fully owned custom GraphQL backend using modern AI coding agents (Google Antigravity and Claude Code). The resulting product can be branded and sold as a managed or self-hosted solution to individual grocery stores and supermarts.

Core value proposition for local businesses: Own a branded ordering + delivery system (customer PWA + delivery boy app + owner dashboard) without paying high monthly SaaS fees to big platforms, while keeping customer data and brand control.

2. Source Repository (Frontend)

Primary Repository

Enatega Single Vendor Delivery Management System
GitHub: **https://github.com/enatega/food-delivery-singlevendor**
Stars: ~675 | License: MIT (frontend) | Backend: Proprietary (we will replace it)

What the repository provides (Open Source):

- Customer App (React Native / Expo) – ordering experience for the grocery store
- Rider / Delivery Boy App (React Native / Expo) – accept orders, navigation, status updates
- Admin Dashboard – manage products, orders, riders, and settings
- White-label ready – easy to brand with the grocery store name and logo
- Apollo Client GraphQL integration already wired in all clients
- Purpose-built for a single store (no multi-vendor complexity)

Important: The official backend/API is proprietary and requires a paid license. This plan deliberately replaces it with our own open, owned backend so we have full white-label freedom and no recurring license cost.

3. Technology Stack

Layer	Technology	Purpose
Frontend (given)	Enatega (React Native + Next.js + Expo)	Customer App, Rider App, Admin Dashboard
Database + Auth + Realtime	**Supabase** (PostgreSQL)	Users, stores, products, orders, riders, zones, RLS, Realtime
GraphQL API Layer	GraphQL Yoga / Apollo Server or Hasura	Must match Enatega's expected queries, mutations & subscriptions
Hosting / Deploy	**Vercel**	Serverless/Edge GraphQL API + frontend hosting if needed
Source Control	**GitHub**	Repos for backend + forked frontend, CI/CD
Payments (India)	**Razorpay**	UPI, Cards, Wallets, COD – replace Stripe/PayPal
Image Storage	**Cloudinary**	Product images, logos, banners (Enatega already supports Cloudinary fields)
AI Coding Agents	**Google Antigravity** + **Claude Code**	Reverse-engineer schema, generate resolvers, migrations, integrations
Maps / Location	Google Maps API (or Mapbox)	Nearby stores, rider tracking, delivery zones

4. High-Level Architecture

All Enatega client applications (Customer, Rider, Store, Admin, Web) communicate exclusively via GraphQL (HTTP + WebSocket for subscriptions). Our custom backend will expose the same GraphQL contract so the existing frontends continue to work with only environment variable changes.

Component	Responsibility
Enatega Frontends	UI + Apollo Client. Point GRAPHQL_URL and WS_GRAPHQL_URL to our API.
Custom GraphQL API (Vercel)	Resolvers that implement Enatega's operations. Auth, business logic, Razorpay, Cloudinary.
Supabase	PostgreSQL tables + Auth (JWT) + Realtime channels + optional Storage.
Razorpay Webhooks	Update order payment status (PAID / FAILED) securely.
Cloudinary	Signed or server-side uploads for product & store images.

Recommended starting mode: Single-vendor / Single-supermart. Enatega already supports a single-vendor mode. This repo is purpose-built for one store – much simpler than multi-vendor.

5. Implementation Roadmap (AI-Assisted)

Phase 0 – Preparation (1–2 days)

- Fork the Enatega repository on GitHub and clone it locally.
- Install and set up Google Antigravity (agentic IDE) and Claude Code.
- Create Supabase project, Vercel account, Razorpay test account, Cloudinary account.
- Extract all GraphQL operations from apollo/queries.js, apollo/mutations.js and subscription files across apps.

Phase 1 – Schema & Database Design (AI-heavy)

- Feed the extracted GraphQL operations into Antigravity / Claude Code.
- Prompt: "Generate a complete GraphQL schema (SDL) that satisfies every query, mutation and subscription used by the Enatega frontends."
- Prompt: "Design a clean PostgreSQL schema for Supabase that supports Users, Stores, Categories, Products, Variations, Orders, OrderItems, Riders, Zones, Addresses, Configuration."
- Create tables + Row Level Security policies in Supabase.
- Generate TypeScript types from the schema.

Phase 2 – Core GraphQL API & Auth

- Scaffold GraphQL Yoga or Apollo Server project (deployable to Vercel).
- Implement authentication resolvers that work with Supabase Auth (or map JWTs).
- Implement highest-priority operations first: profile, restaurant/store list, product catalog, placeOrder, myOrders, order status subscription.
- Wire Supabase client inside resolvers.

Phase 3 – Payments, Images & Real-time

- Integrate Razorpay: create order → return payment details → webhook to mark order PAID.
- Cloudinary upload endpoints / signed upload flow for product images and store logos.

• Real-time order status and rider location using GraphQL subscriptions (backed by Supabase Realtime or WebSockets).
• Basic rider assignment logic for single-store mode.

Phase 4 – Connect Frontends & Polish

• Update environment files in all Enatega apps: GRAPHQL_URL, WS_GRAPHQL_URL, SERVER_URL.
• Test end-to-end: Customer places order → Store accepts → Rider delivers → Status updates in real time.
• White-label branding: logos, colors, app names, domains.
• Turn customer web into a proper installable PWA if not already strong enough.
• Add basic admin features needed for a local supermart (inventory, reports, delivery zones).

Phase 5 – Packaging for Sale

• Create deployment scripts / Docker / one-click style setup documentation.
• Prepare demo instance with sample data.
• Write simple sales one-pager and pricing packages (Setup + Monthly managed hosting).
• Optional later: multi-tenant wrapper if you want to host many single-store clients from one backend.

6. AI Tool Usage Strategy (Antigravity + Claude Code)

Google Antigravity is an agent-first development platform that can plan, write, test and iterate on large pieces of code. Claude Code is excellent for precise, high-quality code generation and refactoring. Use them together:

Task	Recommended Tool & Approach
Extract & catalog all GraphQL ops	Antigravity or Claude – scan entire frontend folders and produce a structured list
Generate GraphQL SDL + TS types	Claude Code for high accuracy schema; Antigravity for large multi-file generation
Supabase SQL migrations + RLS	Claude Code (precise SQL) + Antigravity for iterative testing
Resolver implementation	Antigravity agent for bulk scaffolding; Claude for complex business logic & Razorpay webhooks
Debugging mismatches	Point frontend at local API, capture failing queries, feed errors back to the agent
Documentation & deployment scripts	Either tool – generate README, Vercel config, env examples

Sample high-value prompt for Antigravity / Claude:
“You are building a GraphQL backend that must be drop-in compatible with the Enatega singlevendor frontend. Here are all the queries and mutations extracted from the client. Generate the complete schema, PostgreSQL tables for Supabase, and TypeScript resolvers for the placeOrder and subscriptionOrder operations, integrating Razorpay for payment and Cloudinary for images.”

7. Business Model & Monetization (Selling to Local Businesses)

Target customers: Independent grocery stores, local supermarts, and small local kirana / supermart owners.

Suggested Pricing Packages

Package	What Client Gets	Indicative Price (INR)
Setup + Branding	White-labeled apps + admin, domain, basic training, one-time deployment	■25,000 – ■80,000 one-time

Package	What Client Gets	Indicative Price (INR)
Managed Hosting	You host & maintain the backend + apps, updates, monitoring, support	■3,000 – ■12,000 / month
Self-Hosted License	Full source or Docker image + documentation (client runs on their server)	■40,000 – ■1,20,000 one-time
Add-ons	Custom reports, WhatsApp integration, WhatsApp order alerts, extra training	As quoted

Sales Channels (from earlier research)

- Direct outreach to local grocery / supermart owners (WhatsApp, visits, local Facebook groups).
- Partnerships with digital marketing agencies, IT service providers, and accountants who already serve SMBs.
- Your own simple website + Google ranking for “grocery delivery software [city]”.
- Demo instance that prospects can try with sample data.

8. Key Risks & Mitigations

Risk	Mitigation
GraphQL schema mismatch with frontend	Extract every operation first; implement and test incrementally against real clients.
Scope creep (trying to add multi-store features too early)	Stay focused on single-store. Do not expand to multi-vendor until you have paying clients.
AI-generated code quality / security	Always review critical paths (auth, payments, RLS). Use Claude for security-sensitive code.
Support burden after selling	Offer managed hosting first. Document everything. Charge for support beyond basic.
License / legal of frontend	Enatega frontend is MIT – commercial use and white-labeling are allowed. Keep attribution where required.

9. Success Metrics & Immediate Next Actions

Success looks like:

- A working demo: Customer can place an order on the PWA → Rider sees it → Status updates live → Payment via Razorpay succeeds.
- At least one local grocery/supermart paying for a branded instance within 60–90 days of MVP.
- You fully understand how to reverse-engineer an open-source frontend and replace its backend with AI – a transferable skill for future white-label products (POS, school ERP, coaching systems, etc.).

Immediate Next Actions (This Week)

1. Fork https://github.com/enatega/food-delivery-singlevendor and explore the apollo folders.
2. Create Supabase, Vercel, Razorpay (test), and Cloudinary accounts.
3. Install Google Antigravity and Claude Code; load the frontend repo into the AI environment.
4. Extract and list every GraphQL operation used by the customer and rider apps.
5. Generate the first version of the GraphQL schema and Supabase table definitions with the AI tools.

This plan turns an open-source frontend into a fully owned, white-labelable product using modern AI coding agents. The same process can later be repeated for school ERPs, coaching management systems, POS tools, and other local-business software.

Focus on shipping a single-store MVP first, sell it, learn from real users, then expand.